

DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

Artificial Neuron and Perceptron

How the brain inspired a mathematical model, and how that model learns a decision boundary

Session 2 (3 May 2026) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation, and every slide example is worked out in full, step by step. Colour-coded boxes guide you: the **blue** “intuition” box states the core idea; the **amber** “everyday picture” box ties it to real life; the **green** “worked example” box solves a problem with real numbers; the **red** “watch out” box flags the common mistake; and the **purple** “key takeaway” box is the one sentence to remember.

1 What this session covers

Session 2 opens the *model box*. Session 1 showed that a deep learning system needs a model — a parameterised transformation from inputs to predictions. This session asks: where does the idea of a neuron-like model even come from? It then builds the answer layer by layer: human brain → biological neuron → artificial neuron → perceptron → perceptron learning algorithm → the XOR wall that one perceptron cannot climb. That wall motivates everything that follows in the course.

The session also revisits and re-anchors the four DL ingredients (Slides 3–16), then zooms into the new material from Slide 17 onward.

2 From brain to artificial neuron

2.1 The brain as inspiration

The human brain has roughly 10^{10} neurons, each connected to 10^4 – 10^5 others. A single neuron switches in about 0.001 seconds, yet the brain can recognise a scene in 1 second. That means only about 100 sequential steps are available — far too few if each decision required a long chain of thought. The conclusion: the brain must do its work in *parallel*, with many neurons computing at once.

The intuition

The brain is a massively parallel computer where every processor (neuron) is simple, but the sheer number of processors and connections makes the whole thing extraordinarily powerful. The inspiration for artificial neural networks is not to copy the chemistry but to copy this *structure*: many simple units, heavily interconnected.

★ Everyday picture

A city's traffic flows during rush hour: no single traffic light is clever, but thousands of them, each responding to local conditions, produce fluid city-wide flow. The brain — and a neural network — works the same way: local simplicity, global intelligence through scale.

Where this shows up in ML. Biological plausibility is the *motivation*, not the constraint. Actual deep networks deviate from biology in many ways (real neurons do not use backpropagation). What survives the translation is the structural intuition: layers of simple units, massively parallel, with knowledge encoded in connection strengths.

✓ Key takeaway

The brain does ≈ 100 serial steps in a scene-recognition second; it must be doing enormous amounts of parallel work. That parallelism is what artificial neural networks model.

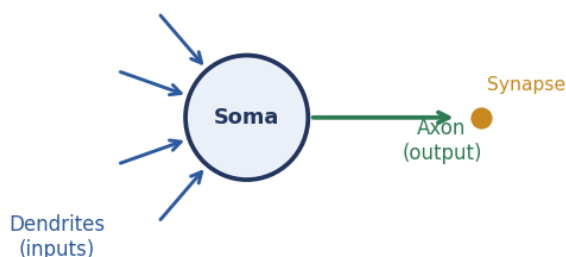
2.2 Biological neuron

A biological neuron has four key parts, and each maps directly onto an artificial neuron's components:

- **Dendrites** — receive incoming signals from other neurons (maps to: the input features x_i).
- **Cell body (soma)** — processes and integrates the incoming signals (maps to: the weighted sum $\sum w_i x_i$).
- **Axon** — transmits the output signal onward (maps to: the output $\hat{y}$).
- **Synapses** — the connection points between neurons, whose strength determines how much influence one neuron has on the next (maps to: the weights w_i).

Biological inspiration → mathematical abstraction

Biological Neuron



Artificial Neuron

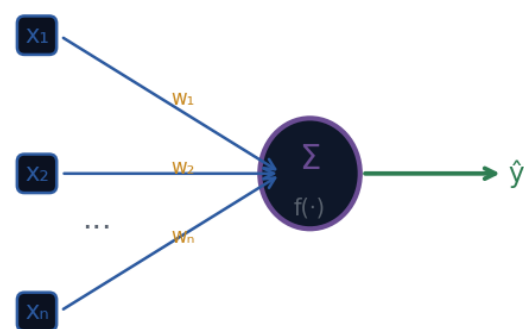


Figure 1: How to read it: left panel is a schematic biological neuron; right panel is the corresponding mathematical abstraction. Follow dendrites → inputs, soma → weighted sum and activation, axon → output.

✓ Key takeaway

Synaptic strength $\leftrightarrow$ weight w_i . The brain stores knowledge in connection strengths; artificial networks store knowledge in weights.

3 The artificial neuron

An **artificial neuron** is a processing element inspired by biology. Like its biological counterpart, it receives signals from other neurons, does some computation, and sends one signal out. Each artificial neuron is interconnected to other neurons, and the knowledge the network learns is stored as the parameters of those connections — the weights.

Formally, neuron N has d input features $x_1, x_2, \dots, x_d$ and d matching weights $w_1, w_2, \dots, w_d$. Weight w_i encodes the *importance* of feature x_i to this neuron's decision. The neuron produces one numerical output $\hat{y}$ by doing two things:

1. **Form the weighted sum.** Each input is multiplied by its weight and the products are summed. This is the **pre-activation**, often called z :

$$z = \sum_{i=1}^d w_i x_i = w_1 x_1 + w_2 x_2 + \dots + w_d x_d. \quad (1)$$

2. **Apply the activation function.** Pass z through a function $f(\cdot)$ to produce the output $\hat{y} = f(z)$. The activation function introduces non-linearity; without it, stacking many layers would collapse to a single linear transformation.

👉 The intuition

The weighted sum is a *vote*: each input casts a vote weighted by how important it is. The activation function is the *decision*: given the total vote, what does this neuron output? A large positive z means many important inputs fired; a large negative z means they mostly did not.

★ Everyday picture

A judge in a talent competition listens to three acts (x_1, x_2, x_3) and scores each. She gives more weight to acts in her specialty. Her final decision — yes/no — is the activation. The weighted sum is her internal score; the threshold is the bar she sets for “yes.”

Where this shows up in ML. Every layer in every neural network in this course is a collection of neurons doing exactly this: weighted sum, then activation. The difference between architectures (CNN, RNN, transformer) is mostly *how* the weights are shared and which inputs connect to which neurons — not the neuron's computation, which stays the same.

✗ Watch out

Weights can be negative. A large negative weight means “if this input is active, suppress the neuron.” Biological inhibitory synapses do the same thing. Do not assume all weights are positive.

✓ Key takeaway

Artificial neuron: weighted sum $z = \sum w_i x_i$, then activation $\hat{y} = f(z)$. Two operations, stacked many times, are the whole of deep learning.

4 Artificial Neural Networks

An **artificial neural network (ANN)** is simply a collection of artificial neurons arranged into layers and connected together. The key properties:

- Many neuron-like **threshold switching units** (neurons with an activation).
- Many **weighted interconnections**: connections can run forward (input to output), recursively (later layers back to earlier ones), or as self-loops.
- **Highly parallel, distributed processing**: many neurons compute simultaneously.
- Emphasis on **tuning weights automatically** through a learning algorithm, rather than hand-setting them.

The intuition

An ANN is a *connectionist machine*: all of the knowledge it has is stored in the pattern of connection weights. There are no explicit rules inside the network, only weighted pathways. Learning means adjusting those weights.

When should you reach for an ANN? The slides list a practical checklist: the input is high-dimensional (raw sensor data, images, audio); the data may be noisy; the target may be discrete, continuous, or a vector; the form of the target function is unknown; and human readability of the result is not needed. Classic examples: speech phoneme recognition, image classification, financial prediction.

Where this shows up in ML. The connectionism principles — intelligence from simple units, knowledge in weights, learning by modifying those weights, parallel distributed processing — are the founding philosophy of the entire field of deep learning.

Watch out

“ANN” is the old name for what the course calls “deep neural network” when it has many layers. A network with one hidden layer is shallow; three or more is deep. The distinction matters for expressive power.

Key takeaway

An ANN = many neurons + weighted connections + a learning algorithm. Knowledge lives in the weights, not in explicit rules.

5 The perceptron

The **perceptron** is the simplest possible artificial neuron — the atom of the course. It was introduced by Frank Rosenblatt in 1958. A perceptron takes n binary or real-valued inputs $x_1, \dots, x_n$ (plus a special **bias** input fixed at 1) and applies one weight to each, sums the result, and passes it through a **step function** (also called a threshold function):

$$z = \sum_{i=0}^n w_i x_i = w_0 \cdot 1 + w_1 x_1 + \dots + w_n x_n, \quad (2)$$

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq \theta, \\ 0 & \text{otherwise,} \end{cases} \quad (3)$$

where θ is the **threshold**. (Equivalently, absorb θ into the bias: $w_0 = -\theta$, so the rule becomes $\hat{y} = 1$ if $z \geq 0$.) The output is binary: the perceptron either *fires* (1) or does not (0).

The intuition

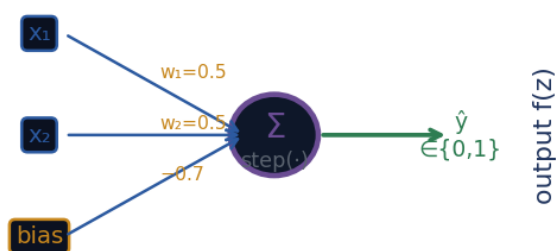
The perceptron is a linear classifier: it draws one straight line (in 2D), one plane (in 3D), or one **hyperplane** (in n D) through the input space, and puts everything on one side in class 1, everything on the other in class 0. One line, one decision.

★ Everyday picture

A bouncer at a club has one rule: “total score ≥ 7 and you’re in.” He checks dress code (weight 3), ID validity (weight 4), and queue time (weight 2). A person who scores 8 gets in (fires); someone who scores 5 does not. The bouncer is a perceptron. He can only apply one rule; he cannot handle “in if tall *or* well-dressed, but not both” (the XOR problem).

The perceptron: weighted sum through a hard threshold

Perceptron structure



Step activation

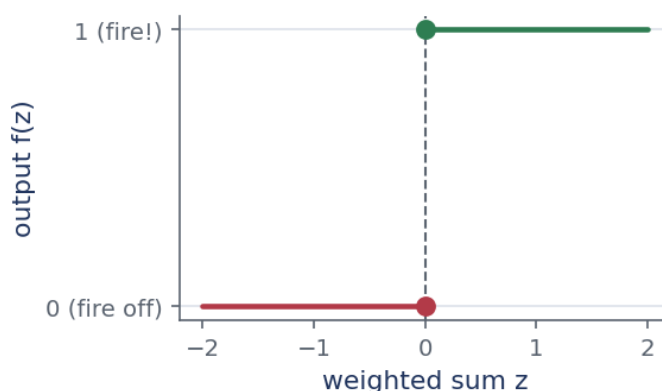


Figure 2: How to read it: left panel is the perceptron’s wiring diagram (inputs x_i , weights w_i , summation node, step activation, binary output). Right panel is the step function: output is 0 for $z < 0$, jumps to 1 at $z = 0$.

Where this shows up in ML. The perceptron is the ancestor of every deep network unit. Replace the step function with a smooth activation (sigmoid, ReLU, tanh) and you have the modern neuron. Stack many such neurons and you have a deep network. Understanding the perceptron is understanding the atom.

✗ Watch out

The bias weight w_0 is crucial. Without it, the decision hyperplane is forced to pass through the origin — a severe restriction. Always include the bias.

✓ Key takeaway

Perceptron = weighted sum + step function = a single linear decision boundary. One straight cut through the input space, and that’s all.

6 Perceptron for logic gates

Any linear decision can be represented by a Boolean (logic gate) equation. The perceptron can represent any gate whose truth table is *linearly separable* — meaning a straight line can separate the 1-outputs from the 0-outputs. AND and OR are linearly separable; XOR is not.

AND and OR: a single straight line separates both

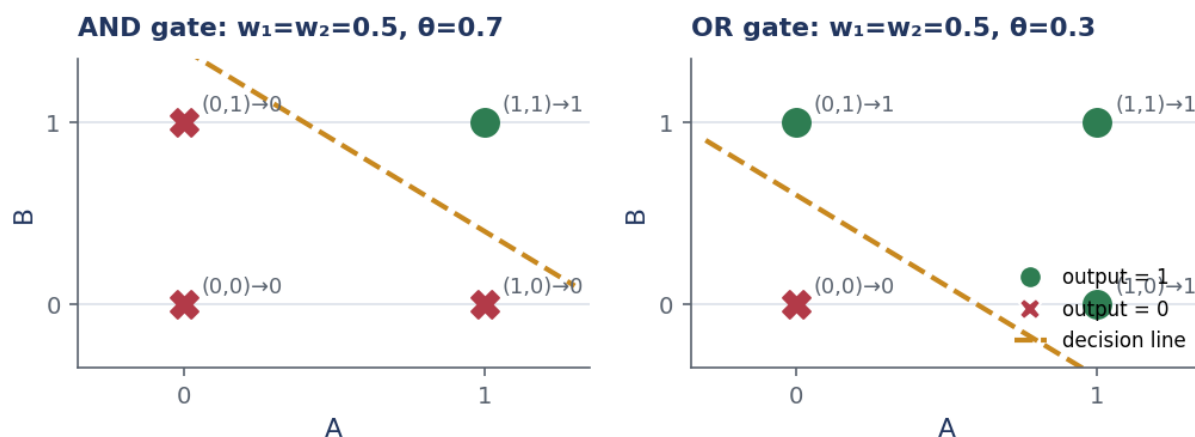


Figure 3: How to read it: the dashed amber line is the perceptron’s decision boundary. AND gate (left): all 1-outputs (green circles) sit above the line, all 0-outputs (red X) below. OR gate (right): the same story with a different line.

Worked example: AND gate — finding weights by inspection

The AND gate fires (output 1) only when *both* inputs are 1. Truth table: $(0,0) \rightarrow 0$; $(0,1) \rightarrow 0$; $(1,0) \rightarrow 0$; $(1,1) \rightarrow 1$. We want $w_1A + w_2B + w_0 \geq 0$ when $(A,B) = (1,1)$ and < 0 otherwise.

Step 1. Try $w_1 = w_2 = 0.5$, $\theta = 0.7$ (i.e. $w_0 = -0.7$).

Step 2. Check all four rows.

$$\begin{aligned} (0,0): \quad z &= 0.5(0) + 0.5(0) - 0.7 = -0.7 < 0 \Rightarrow \hat{y} = 0. \quad \checkmark \\ (0,1): \quad z &= 0.5(0) + 0.5(1) - 0.7 = -0.2 < 0 \Rightarrow \hat{y} = 0. \quad \checkmark \\ (1,0): \quad z &= 0.5(1) + 0.5(0) - 0.7 = -0.2 < 0 \Rightarrow \hat{y} = 0. \quad \checkmark \\ (1,1): \quad z &= 0.5(1) + 0.5(1) - 0.7 = +0.3 \geq 0 \Rightarrow \hat{y} = 1. \quad \checkmark \end{aligned}$$

Step 3. All rows match. Parameters: $w_1 = 0.5$, $w_2 = 0.5$, $w_0 = -0.7$.

Step 4. Sense-check. The decision line $0.5A + 0.5B = 0.7$, or $A + B = 1.4$, separates $(1,1)$ from $(0,0), (0,1), (1,0)$. Geometrically the only “both-on” corner is above the line.

OR gate (exercise answer). Try $w_1 = w_2 = 0.5$, $\theta = 0.3$ ($w_0 = -0.3$): $(0,0)$ gives $z = -0.3 < 0$; $(0,1)$ gives $z = +0.2 \geq 0$; $(1,0)$ gives $z = +0.2 \geq 0$; $(1,1)$ gives $z = +0.7 \geq 0$. All correct.

Watch out

There are infinitely many correct weight settings for AND and OR (any line that separates the classes works). The weights found by inspection are just one valid answer; the perceptron learning algorithm finds a solution automatically.

✓ Key takeaway

AND and OR are linearly separable, so a single perceptron (one line) can represent them exactly. The weights encode which line to draw.

7 The perceptron learning algorithm

Finding weights by inspection works for tiny examples, but fails in high dimensions. The **perceptron learning algorithm** finds the weights automatically from data. It is a **gradient-free online update rule**: it processes one example at a time and corrects only when it makes a mistake.

📖 The intuition

The algorithm says: if you got it right, do nothing; if you predicted 1 but the answer was 0, nudge the weights down; if you predicted 0 but the answer was 1, nudge the weights up. Repeat until no mistakes are made.

The update rule for each training example (x, t) :

$$e = t - \hat{y} \quad (\text{error: } +1, 0, \text{ or } -1), \quad (4)$$

$$w_i \leftarrow w_i + \eta e x_i \quad \text{for all } i, \quad (5)$$

where $\eta > 0$ is the **learning rate** (step size of the correction). When $e = 0$ (correct prediction), nothing moves. When $e \neq 0$, the weights shift in the direction that would have produced the correct output.

The **perceptron convergence theorem** guarantees that if the data is linearly separable, the algorithm converges in finite steps — it will eventually find a set of weights that classifies everything correctly.

🔗 Worked example: NOT gate — running the learning algorithm

The NOT gate has one input x and one output $t = 1 - x$: $(x = 0) \rightarrow t = 1$; $(x = 1) \rightarrow t = 0$. Start with $w = 0$, bias $w_0 = 0$, learning rate $\eta = 1$. Threshold: $\hat{y} = 1$ if $wx + w_0 \geq 0$ else 0. (Tie goes to 1.)

Step 1. Epoch 1, example $(x = 0, t = 1)$. $z = 0(0) + 0 = 0 \geq 0 \Rightarrow \hat{y} = 1$. Error $e = 1 - 1 = 0$. No update.

Step 2. Epoch 1, example $(x = 1, t = 0)$. $z = 0(1) + 0 = 0 \geq 0 \Rightarrow \hat{y} = 1$. Error $e = 0 - 1 = -1$. Update: $w \leftarrow 0 + 1 \cdot (-1) \cdot 1 = -1$; $w_0 \leftarrow 0 + 1 \cdot (-1) = -1$. Errors this epoch: 1.

Step 3. Epoch 2, example $(x = 0, t = 1)$. $z = (-1)(0) + (-1) = -1 < 0 \Rightarrow \hat{y} = 0$. Error $e = 1 - 0 = +1$. Update: $w \leftarrow -1 + 1 \cdot 1 \cdot 0 = -1$; $w_0 \leftarrow -1 + 1 \cdot 1 = 0$. Errors: 1.

Step 4. Epoch 2, example $(x = 1, t = 0)$. $z = (-1)(1) + 0 = -1 < 0 \Rightarrow \hat{y} = 0$. Error $e = 0 - 0 = 0$. No update.

Step 5. Epoch 3, example $(x = 0, t = 1)$. $z = (-1)(0) + 0 = 0 \geq 0 \Rightarrow \hat{y} = 1$. Error = 0. No update.

Step 6. Epoch 3, example $(x = 1, t = 0)$. $z = (-1)(1) + 0 = -1 < 0 \Rightarrow \hat{y} = 0$. Error = 0. No update. **Converged!**

Final parameters: $w = -1$, $w_0 = 0$. Check: $\hat{y}(0) = f(-1 \cdot 0 + 0) = f(0) = 1$; $\hat{y}(1) = f(-1 + 0) = f(-1) = 0$. Both correct. The negation is captured by the negative weight: input 1 gives $z = -1$ (fires off); input 0 gives $z = 0$ (fires on at the threshold).

Where this shows up in ML. The perceptron rule is a special case of **stochastic gradient descent** with a non-differentiable step activation. Modern training replaces the step with smooth activations and uses backpropagation to compute gradients — but the spirit (see the error, adjust the parameters, repeat) is identical.

Perceptron learning trace: NOT gate ($\eta = 1$)

Epoch	w (weight)	w_0 (bias)	Errors corrected
1	-1.0	-1.0	1
2	-1.0	0.0	1
3	-1.0	0.0	0 ← converged ✓
4	-1.0	0.0	0 ← converged ✓
5	-1.0	0.0	0 ← converged ✓
6	-1.0	0.0	0 ← converged ✓

Rule: $w \leftarrow w + \eta(t - \hat{y}) \cdot x$ and $w_0 \leftarrow w_0 + \eta(t - \hat{y})$

Figure 4: How to read it: each row is one epoch of the NOT-gate trace. Watch the weight and bias change only when errors occur; once both rows produce zero errors, the algorithm stops. The table records the exact numbers from the worked example.

✗ Watch out

If the data is *not* linearly separable, the perceptron learning algorithm never converges — it cycles forever. That is not a bug in the algorithm; it is a fact about the data. The XOR problem is the canonical example.

✓ Key takeaway

Perceptron update: $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$. Correct predictions leave weights alone; mistakes pull the boundary toward the correct side. Converges in finite steps *if and only if* the data is linearly separable.

8 The XOR problem

The **XOR** (exclusive-OR) gate fires when the two inputs *differ*: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$. Plot these four points in 2D with colour showing the output. The two 1-outputs sit at opposite corners of the unit square; the two 0-outputs sit at the other two corners. *No single straight line can separate them.*

🧠 The intuition

XOR is the first problem that a single perceptron cannot solve, and that is not a failure of the learning algorithm — it is a fundamental limitation of linear models. You would need to bend, fold, or curve the boundary, and a straight line cannot do that.

★ Everyday picture

Imagine two light switches on a circuit: the light is on when exactly one switch is flipped, and off when both are the same. You want a rule that says “on” for $(0,1)$ and $(1,0)$ and “off” for $(0,0)$ and $(1,1)$. Try to draw one straight line separating the on-points from the off-points on a 2×2 grid. You cannot — the on-points are diagonally opposite.

Worked example: why no line separates XOR — three failed attempts

Step 1. Try line $A + B = 1$ (diagonal). $(0, 1)$: $\text{sum} = 1 \Rightarrow$ border (could go either way). $(1, 0)$: same. $(0, 0)$: $\text{sum} = 0$ (class 0). $(1, 1)$: $\text{sum} = 2$ (class 1). But we need $(1, 1)$ to be class 0, not 1. Fails.

Step 2. Try line $B = 0.5$ (horizontal mid-line). $(0, 0)$ and $(1, 0)$ get class 0 (below). $(0, 1)$ and $(1, 1)$ get class 1 (above). Truth: $(1, 1)$ must be 0. Fails.

Step 3. Any line? Formal proof by contradiction: the four XOR points are at the vertices of a square. Any line that separates the two diagonals also separates some 1-output from the other. No hyperplane in 2D can accomplish the task. The data is not linearly separable.

The fix: add a *hidden layer*. With two decision boundaries instead of one, the region “exactly one input is 1” can be carved out. Two perceptrons in a hidden layer feed a third output perceptron, and together they solve XOR. This is the birth of the multilayer network.

The XOR problem: why one perceptron is not enough

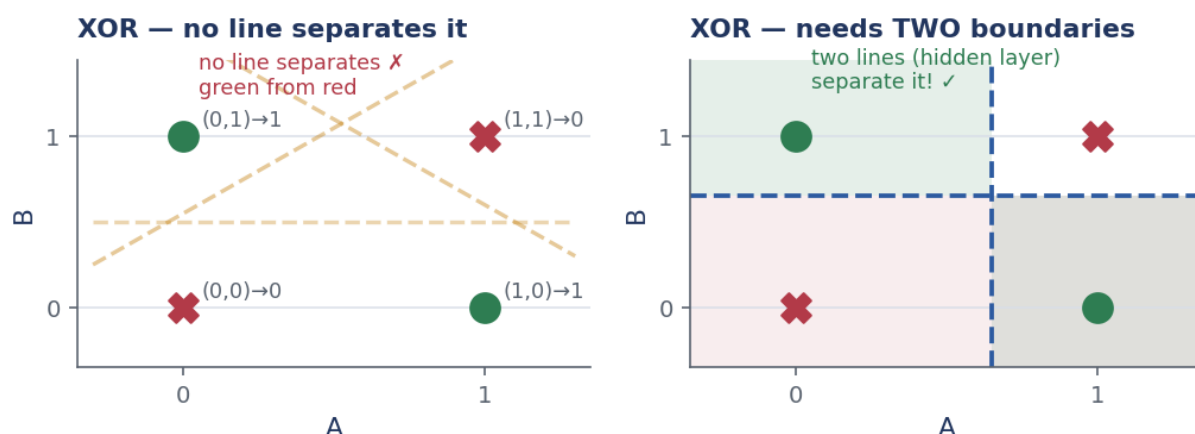


Figure 5: How to read it: left panel — three candidate lines are drawn on the XOR scatter; every one misclassifies at least one point. Right panel — two boundaries (from a hidden layer) successfully separate the classes.

Where this shows up in ML. Minsky and Papert’s 1969 book *Perceptrons* proved formally that a single perceptron cannot solve XOR, triggering the first “AI winter.” The response (the multilayer perceptron + backpropagation) is what Session 5 covers. XOR is the canonical motivating example for depth.

✗ Watch out

The failure on XOR is not a failure of learning — the learning algorithm would converge just fine on linearly separable data. The failure is *representational*: one linear boundary cannot express the XOR function, no matter how long you train.

✓ Key takeaway

XOR is not linearly separable: no single perceptron (one straight boundary) can classify it. The solution is a *hidden layer* giving two boundaries — the birth of the deep network.

9 Linear separability and the perceptron’s scope

Linearly separable data means two sets of points can be separated by an $(n - 1)$ -dimensional hyperplane in n -dimensional space (a line in 2D, a plane in 3D). A perceptron converges if and only if the data is linearly separable. There are infinitely many valid separating hyperplanes; the learning algorithm finds one of them.

The slides summarise the perceptron as a complete DL framework for this restricted case:

- **Data:** truth table or labelled examples.
- **Model:** perceptron (inputs x_i , target label t).
- **Objective:** deviation between desired output t and computed output $\hat{y}$.
- **Learning algorithm:** perceptron learning rule.
- **Challenge:** find the $n + 1$ parameters $w_0, w_1, \dots, w_n$ automatically.
- **Solution:** run the perceptron learning algorithm.

✓ Key takeaway

The perceptron is a complete learning system for linearly separable binary classification. When the data is not linearly separable, we need more layers.

10 Recap and what comes next

One paragraph to carry away. The brain's massively parallel structure inspired artificial neurons: weighted sum $z = \sum w_i x_i$, then activation $f(z)$. Stacking many such neurons gives an ANN. The perceptron (Rosenblatt, 1958) is the simplest such unit: binary step activation, one linear decision boundary. It can represent AND, OR, NOT, NOR, NAND — all linearly separable gates — but *not* XOR, which is non-linearly separable. The update rule $w_i \leftarrow w_i + \eta e x_i$ (where $e = t - \hat{y}$) converges in finite steps on linearly separable data. XOR's failure motivates hidden layers — the subject of Session 5 and beyond.

✓ Key takeaway

Biological neuron $\rightarrow$ artificial neuron (weighted sum + activation) $\rightarrow$ perceptron (step activation, one boundary) $\rightarrow$ learning algorithm (error correction) $\rightarrow$ XOR wall $\rightarrow$ hidden layers. That chain is the spine of the next several sessions.

Further reading. Zhang et al. *Dive into Deep Learning*, Chapter 1; Goodfellow et al. *Deep Learning*, Chapters 1–6; Rosenblatt (1958); Minsky & Papert (1969).